

ETHICAL HACKING FINAL
COSC 6840
HARDWARE SECURITY ASSESSMENT

Kevin Umba

12/9/2025

AGENDA

- Executive Summary
- Methodology
- Findings
- Risks and Fixes
- Script and Demo
- Recommendations

EXECUTIVE SUMMARY

- **Objective:** ThanosTech LLC - The objective is to uncover weaknesses, of IoT devices and recommend fixes.
- **Major Findings:** Unprotected UART Debug Shell & weak password, exposed flash memory.
- **Devices:** STM32F103C8T6 Microcontroller (MCU), Winbond W25Q64JV External SPI Flash
- **Recommended Next step:** **strongly advice** to recall devices as these vulnerabilities are easily actionable and may lead to **massive intellectual property and revenue loss**.

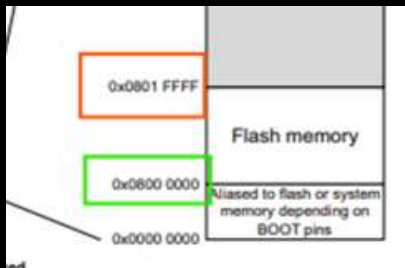
Risk	Description	Device	Mitigation
High	Unprotected UART peripheral exposing development information, credentials	MCU	Disable or solder UART pins Use secure bootloader
High	Weak password exposed in debug shell	MCU	Salted hashing and password complexity Rotate & Revoke passwords
High	Flash memory exposed potentially leaking firmware	MCU & Winbond	Use RDP1

METHODOLOGY

OSINT & Device Info

Collected device connection layout, memory mapping, UART data streams via manufacturer datasheets

-	PE14	I/O	FT	PE14	-	TIM1_CH4
-	PE15	I/O	FT	PE15	-	TIM1_BKIN
-	PB10	I/O	FT	PB10	I2C2_SCL/ USART3_TX ⁽⁹⁾	TIM2_CH3
-	PB11	I/O	FT	PB11	I2C2_SDA/ USART3_RX ⁽⁹⁾	TIM2_CH4
18	VSS_1	S	-	VSS_1	-	-
19	VDD_1	S	-	VDD_1	-	-



UART Data Streams (Finding 1&5)

- Tools: FT232 USB-UART adapter, screen, usbipd
- Investigated default USART pins at different baud rates to access debug and secret data streams.



```
info : Information about this shell
get_uart_flag : Retrieve the flag for connecting to UART
get_secret_hash : Retrieve the hashed password for the 'submit_password' command
submit_password : Submit the secret password to get a flag
submit_memory : Submit the password from internal memory to get a flag
flag_to_proof : Submit a flag to generate your unique proof for submission
toggle_led : Toggle the on-board LED
uart~:$ get_secret_hash
The following is the hashed password for the 'secret' area. Can you crack it?
Hash: 37c3bb681afc98e2df6f48d1576071add74blad2
uart~:$
```

===== Mini UART Shell =====

```
FLAG{DOUBLE_THE_UART_DOUBLE_THE_FUN}FLAG{DOUBLE_THE_UART_DOUBLE_THE_FUN}FLAG{DOUBLE_TH
E_UART_DOUBLE_THE_FUN}FLAG{DOUBLE_THE_UART_DOUBLE_THE_FUN}FLAG{DOUBLE_THE_UART_DOUBLE_
THE_FUN}
```

METHODOLOGY

Password Extraction (Finding 2)

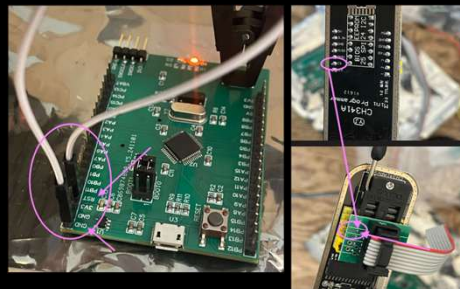
- Tools: John The Ripper, rockyou list
- Discovered the password in the unauthenticated UART debug shell. The hash was easily reversible

```
(laracinedekalite@kalithevert)~/Documents/ethical_hacking/final_project
$ john --wordlist=/usr/share/wordlists/rockyou.txt encoded.txt

Warning: detected hash type "Raw-SHA1", but the string is also recognized as "Raw-SHA1-AxCrypt"
Use the "--format=Raw-SHA1-AxCrypt" option to force loading these as that type instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "Raw-SHA1-Linkedin"
Use the "--format=Raw-SHA1-Linkedin" option to force loading these as that type instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "ripemd-160"
Use the "--format=ripemd-160" option to force loading these as that type instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as "hmac-sha1"
Use the "--format=hmac-sha1" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 256/256 AVX2 8x])
Warning: no OpenMP support for this hash type, consider --fork=8
Press 'q' or Ctrl-C to abort, almost any other key for status
MARQUETTE (?)
ig 0:00:00:00 DONE (2025-11-19 17:37) 5.000g/s 5239Kp/s 5239Kc/s 5239Kc/s MARRA
NO_MARNIE
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords reliably
Session completed.
```

Flash Memory Extraction (Finding 3&4)

- Tools: ST-Link V2, CH341A, openOCD, telnet, flashrom, Hexed.it
- Used the debugging hardware and the memory mapping in the datasheet to perform the flash dump.



```
00000000 01 20 01 02 0f 13 14 20 14 00 03 13 20 10 01 13 g about this pas
000058C8 73 77 6F 72 64 2E 20 43 61 6E 20 79 6F 75 20 64 sword. Can you d
000058D0 75 6D 70 20 69 74 20 66 72 6F 6D 20 6D 79 20 69 ump it from my i
000058E0 6E 74 65 72 6E 61 6C 20 6D 65 6D 6F 72 79 3F 0D nternal memory?
000058F0 00 00 00 00 49 6E 76 61 6C 69 64 20 73 79 6E 74 ...Invalid synt
00005900 61 78 21 20 45 78 70 65 63 74 65 64 3A 20 27 73 ax! Expected: 's
00005910 75 62 6D 69 74 5F 6D 65 6D 6F 72 79 20 58 56 41 ubmit_memory [VA
00005920 4C 55 45 5D 27 0D 00 00 43 6F 72 72 65 63 74 21 LUE]'. ...Correct!
00005930 20 46 4C 41 47 70 43 4F 4E 53 49 44 45 52 5F 4D FLAG(CONSIDER_M
00005940 45 5F 44 45 42 55 47 47 45 44 7D 00 00 00 00 00 E_DEBUGGED):....
00005950 50 6C 65 61 73 65 20 70 72 6F 76 69 64 65 20 61 Please provide a
00005960 20 66 6C 61 67 20 74 6F 20 67 65 6E 65 72 61 74 flag to generat
00005970 65 20 61 20 70 72 6F 66 2E 00 00 49 6E 76 61 e a proof...Inva
00005980 6C 69 64 20 73 79 6E 74 61 78 21 20 45 78 70 65 lid syntax! Expe
00005990 63 74 65 64 3A 20 27 66 6C 61 67 5F 74 6F 5F 70 cted: 'flag_to_p
000059A0 72 6F 6F 66 20 58 46 4C 41 47 5D 27 0D 00 00 00 roof [FLAG]....
000059B0 38 38 35 00 50 72 6F 6F 66 3A 20 25 73 0D 0A 00 885.Proof: %s...
```

```
kevin@kevin@marquette:~/ethical_hacking/final$ flashrom -p ch341a_spi -c "W25Q64JV-Q" -r dump.bin
flashrom unknown on Linux 6.6.87.2-microsoft-standard-WSL2 (x86_64)
flashrom is free software, get the source code at https://flashrom.org

Using clock_gettime for delay loops (clk_id: 1, resolution: 1ns).
Found Winbond flash chip "W25Q64JV-Q" (8192 kB, SPI) on ch341a_spi.

This flash part has status UNTESTED for operations: WP
The test status of this chip may have been updated in the latest development
version of flashrom. If you are running the latest development version,
please email a report to flashrom@flashrom.org if any of the above operations
work correctly for you with this flash chip. Please include the flashrom log
file for all operations you tested (see the man page for details), and mention
which mainboard or programmer you tested in the subject line.
Thanks for your help!
Reading flash... done.
```

FINDINGS & FLAGS

MCU & Winbond

Finding	Impact	Risk Score	Mitigation
Exposed UART Debug Interface: Finding 1&5	The exposed UART peripherals are well known attack vectors and are cheap and easy to exploit [1]. This could reveal intellectual property and configurations.	High	Disable UART in production or headers. Use secure bootloader for authentication and block code execution. [2]
Weak Password Policy and Storage: finding 2	Password is susceptible to brute force The password was exposed on unauthenticated UART. The hash is unsalted and the plaintext is easily guessable.	High	Adopt complex password requirements Use salted hashing (bcrypt) Revoke credentials and removing from publicly available interfaces [3].
Unprotected Flash Memory: findings 3 & 4	May expose firmware, intellectual property. SWD and unencrypted external flash memory are the most basic attacks for IoT devices. STM32 suite does not support hardware encryption, making it more vulnerable. [4]	High	If SWD is needed for operation, consider enabling readout protection at level 1 to prevent access to memory. [4]

Flags & Proof

Finding	Proof	Flag
1	HKOlzW)LmTW5KMUaVVCSm[Nc]QSaSf)H\	FLAG{I'M_RX'ING_WHAT_YOU'RE_TX'ING}
2	HKOlzQCMm[NcaB`CBYaLS	FLAG{CAN_YOU_CRACK_ME}
3	HKOlzQQMaKCST^[G^RGAcIFSF]	FLAG{CONSIDER_ME_DEBUGGED}
4	HKOlzaYDSV^aYDSV^[GL]THSUI	FLAG{SWEET_SWEET_MEMORIES}
5	HKOlzRQTPNDmVGSaTOTSmFNcDKSaSVG^TWM,	FLAG{DOUBLE_THE_UART_DOUBLE_THE_FUN}

RECOMMENDATION(S)/CONCLUSION

- Findings: critically vulnerable to simple, low cost and quick attacks.
- Impact: peripherals and flash memory expose internal data, firmware, intellectual property, resulting in major revenue loss.
- Recommended next steps:
 - **Strongly consider recalling all affected products** as they expose intellectual property and internal data.
 - Protect sensitive interfaces with authentication secure bootloader, remove or prevent reads on UART, SWD and SPI peripherals, enable readout protection and strong password policies.

REFERENCES

1. Praetorian. (2021, June 29). *Why are JTAG and UART still effective attack vectors for IoT devices?* Praetorian. <https://www.praetorian.com/blog/why-are-jtag-and-uart-still-effective-attack-vectors-for-iot-devices/>
2. Particle. (n.d.). *IoT secure boot*. Particle. <https://www.particle.io/iot-guides-and-resources/iot-secure-boot/>
3. National Institute of Standards and Technology. (2024). *CVE-2024-3263*. NVD. <https://nvd.nist.gov/vuln/detail/CVE-2024-3263>
4. STMicroelectronics. (2020). *AN5156: Introduction to security for STM32 MCUs*. https://www.st.com/resource/en/application_note/an5156-introduction-to-security-for-stm32-mcus-stmicroelectronics.pdf